GUIDE DE DÉMONSTRATION

Guide des 3 démonstrations — Concert Agent

Préparation, trois scénarios pas à pas et checklist pour l'oral.

CA

Concert Agent · Octobre 2026

Objectif : montrer en quelques minutes que l'application est un vrai agent, qu'elle utilise de vraies données et qu'elle demande confirmation avant d'écrire.

Durée : 6 à 8 minutes. **Version courte :** 2 minutes (démo 1 + démo 3).

Ce que le jury doit retenir

1. **Le LLM ne devine pas les concerts** : il choisit un tool, Python l'exécute, puis le résultat revient au LLM.
2. **L'agent sait dire « je n'ai rien trouvé »** au lieu d'inventer une date ou une salle.
3. **Aucune écriture sans accord** : le favori n'est enregistré qu'après un clic sur **Oui**.

---

1. Choisir le mode de démonstration

Mode local (recommandé)

1. Double-cliquer sur `run.bat`.
2. Ouvrir http://127.0.0.1:5000.
3. Garder la fenêtre noire ouverte pendant toute la démonstration.

Le mode local permet de contrôler la base, de recommencer proprement et d'éviter le réveil lent de Render.

Mode en ligne (secours)

Ouvrir https://concert-agent.onrender.com.

Le service gratuit peut mettre environ une minute à se réveiller. Sa base est remise à zéro après un redémarrage, mais les favoris déjà présents ne peuvent pas être effacés en masse depuis l'interface : pour une démonstration propre, préférer le mode local.

Vérifications avant de commencer

- [ ] La page s'ouvre et le chat est visible.
- [ ] Le badge LLM indique `gemini-3.5-flash-lite` ou `mode règles`.
- [ ] Le badge Ticketmaster indique `clé active` (ou les résultats de démo sont acceptés).
- [ ] Spotify est connecté si tu veux montrer l'import (facultatif).
- [ ] Les favoris et l'historique sont vides.

Pour repartir d'une base propre en local :

```
.venv\Scripts\python.exe -c "import db; [db._exec('DELETE FROM ' + t) for t in
('favoris','pending_actions','interactions','tool_traces','resultats_cache')]"
```

Cela supprime les favoris, l'historique et les actions en attente, mais conserve les artistes suivis et les préférences.

Réglages de confort

- Ouvrir un seul onglet sur l'application.
- Zoom à 100 %.
- Fermer les notifications et les autres onglets bruyants.
- Garder ce guide ouvert sur un second écran ou une feuille.

---

2. Déroulé minuté

Temps	Action	Ce que tu dis
0:00	Montrer la page d'accueil	« Voici un agent qui cherche de vrais concerts et demande la permission avant d'enregistrer. »
0:30	Démo 1	« Il consulte mes préférences, puis interroge Ticketmaster. »
2:00	Montrer les traces	« Regardez : le LLM a choisi ce tool, Python l'a exécuté. »
2:30	Démo 2	« Il sait aussi dire qu'il n'a rien trouvé. »
3:30	Démo 3	« Là, il prépare une écriture, mais il ne l'exécute pas encore. »
5:00	Cliquer sur Oui, puis ouvrir la fiche du favori	« Le favori est en base, et la fiche montre toutes les informations. »
6:00	Conclusion	« L'agent ne devine pas : il cherche, il trace, et il demande avant d'agir. »

Démo 0 (optionnelle) — Import Spotify

Préparation

1. Le dashboard Spotify doit contenir `http://127.0.0.1:5000/callback` dans **Redirect URIs**.
2. Le compte doit être ajouté dans **User Management** (5 comptes maximum en mode développement).
3. Cliquer sur **Se connecter avec Spotify**, accepter les scopes, puis revenir sur l'application.

Ce qui doit se passer

- Le badge passe à `Spotify : connecté`.
- Le profil Spotify apparaît en haut à droite.
- Les top artistes et les écoutes récentes sont ajoutés aux artistes suivis (30 maximum, sans doublons).

- Le bouton **Importer mes artistes** permet de relancer l'import.

Phrase d'explication

> « Spotify sert à importer les artistes réellement écoutés. La recherche de concerts, elle, reste faite par Ticketmaster. »

Démo 1 — Recherche avec préférences

But

Montrer que l'agent utilise les préférences et un tool de recherche.

Message à envoyer

> Trouve un concert de mes artistes préférés en Auvergne-Rhône-Alpes le mois prochain.

Ce que tu dois voir

1. L'agent appelle `consulter_preferences`.
2. Il appelle `rechercher_concerts` avec la région `ARA` et la période `mois_prochain`.
3. Il répond avec les concerts trouvés par Ticketmaster.
4. Chaque concert affiche : artiste, date, heure, lieu, prix et lien de billetterie.
5. Exemple de résultat réel observé : **Gims — LDLC Arena, Décines**, les 02/11/2026 et 03/11/2026.
6. Si Ticketmaster ne trouve rien, l'application utilise des données de démonstration et l'indique clairement.

Si le résultat change

Ticketmaster peut modifier ses données. Le nombre de concerts ou les dates peuvent donc varier. Ce qui compte :

- un tool `rechercher_concerts` a bien été appelé ;
- les informations viennent de Ticketmaster ou sont étiquetées `demo` ;
- aucune date n'est inventée.

Preuve à montrer

- Les traces `consulter_preferences` et `rechercher_concerts`.
- Le badge `Ticketmaster : clé active`.
- La source `ticketmaster` ou l'avertissement « Données de démonstration ».

Phrase à dire

> « L'agent ne connaît pas les concerts : il a appelé un tool, et c'est le résultat du tool qui construit la réponse. »

Démo 2 — Aucun résultat sans invention

But

Montrer que l'agent distingue une absence de résultat d'un concert inventé.

Message à envoyer

> Je veux voir un concert à Paris demain, max 5 €.

Ce que tu dois voir

1. L'agent cherche réellement avec les filtres Paris + demain + 5 €.
2. Aucun concert ne correspond.
3. Il répond :

> Je ne trouve aucun concert correspondant à ces critères. Je ne peux pas inventer de dates ni de salles.

Si un concert apparaît quand même

Cela peut arriver si Ticketmaster possède un événement gratuit ou à moins de 5 €. Dans ce cas, dire :

> « Il a trouvé un vrai événement qui respecte les critères. Pour montrer l'absence de résultat, je resserre la demande. »

Puis envoyer :

> Je veux voir Gims à Paris demain.

Preuve à montrer

- La trace `rechercher_concerts` avec le statut `succes` mais zéro concert.
- L'absence de date ou de salle inventée dans la réponse.

Phrase à dire

> « Un agent fiable, c'est aussi un agent qui sait dire qu'il n'a rien trouvé. »

Démo 3 — Écriture avec confirmation

But

Montrer qu'une écriture est bloquée tant que l'utilisateur n'a pas confirmé.

Message à envoyer

> Ajoute le concert de Gims du 2 novembre 2026 à Décines à mes favoris.

Ce que tu dois voir

1. L'agent cherche le concert.
2. Il prépare l'action `ajouter_favori`.
3. Il affiche une demande de confirmation :

> Je vais ajouter « Gims — 02/11/2026 à Decines Charpieu (LDLC ARENA) » à tes favoris. Confirmer ?

4. **Ne pas cliquer tout de suite sur Oui.**
5. Montrer que le favori n'est pas encore dans la colonne « Favoris ».
6. Cliquer sur **Oui, confirmer**.
7. L'agent exécute le tool et répond :

> C'est fait : « Gims — 02/11/2026 à Decines Charpieu » a été ajouté à tes favoris.

8. Le favori apparaît dans la colonne « Favoris ».
9. Cliquer sur le favori : la fiche affiche l'artiste, la date, l'heure, la salle, la ville, la région, le pays, le prix, la source, la date d'ajout et le lien de billetterie lorsqu'il existe.

Variante à montrer

Recommencer avec un autre concert, puis cliquer sur **Non, annuler**. Le favori n'est pas ajouté et la trace indique `refuse`.

Si Gims n'est pas disponible

Envoyer :

> Ajoute le concert de Gims à Lyon à mes favoris.

Le concert vient alors du jeu de démonstration, mais le mécanisme de confirmation reste identique. Le préciser honnêtement :

> « Ici, Ticketmaster ne renvoie pas ce concert : l'application utilise la donnée de démonstration, clairement signalée. La confirmation, elle, est bien réelle. »

Phrase à dire

> « Le tool d'écriture est bloqué tant que l'utilisateur n'a pas confirmé. Ce n'est pas une consigne donnée au LLM : c'est le code Python qui impose la pause. »

Démo 4 (optionnelle) — Historique et traces

Message à envoyer

> Montre-moi l'historique de ce que tu as fait.

Ce qui doit se passer

- L'agent appelle `historique_agent`.
- Il affiche les dernières demandes, réponses et traces.
- Le panneau « Traces de l'agent » de chaque message montre les tools, arguments, statuts et durées.

Phrase à dire

> « On peut vérifier ce que l'agent a réellement fait, pas seulement ce qu'il raconte. »

Plan B si un service ne répond pas

Le badge indique « mode règles »

C'est le mode de secours sans LLM. Il montre les tools et la confirmation, mais les décisions sont prises par des règles Python. Dire :

> « Le LLM est indisponible, donc l'application bascule sur son mode de secours. Les tools et les garde-fous restent identiques. »

Pour activer le LLM : renseigner `GEMINI_API_KEY` dans `.env`, puis redémarrer.

Ticketmaster ne renvoie rien

L'application affiche des données de démonstration avec un avertissement. Ne pas les présenter comme réelles.

Pas d'internet pendant l'oral

Lancer en local avec `MODE_DEMO=1` dans `.env`, puis redémarrer. L'application fonctionne avec SQLite et les données de démonstration, sans Ticketmaster ni Spotify. Le LLM distant peut aussi être indisponible : le mode règles prend le relais.

Spotify ne se connecte pas

Ce n'est pas central. Dire :

> « Spotify sert seulement à importer les artistes. La démonstration principale fonctionne sans lui. »

Le port 5000 est déjà utilisé

```
netstat -ano | findstr :5000
taskkill /PID <PID> /F
```

`python` n'est pas reconnu

Utiliser `run.bat` ou :

```
py -3 -m venv .venv
.venv\Scripts\python.exe app.py
```

Spotify renvoie `INVALID_CLIENT` ou `redirect_uri mismatch`

L'URI doit être identique au caractère près dans le dashboard Spotify. En local : `http://127.0.0.1:5000/callback`.

Spotify renvoie « User not registered in the Developer Dashboard »

Ajouter le compte dans **User Management**. Le mode développement est limité à 5 comptes.

Questions probables du jury

Question	Réponse courte
« Est-ce vraiment un agent ou un chatbot ? »	Le LLM choisit un tool ; Python l'exécute ; le résultat revient au LLM ; la trace le prouve.
« Comment empêchez-vous le LLM d'inventer ? »	Les concerts viennent des tools, le prompt l'interdit et `ajouter_favori` vérifie que le concert existe dans la dernière recherche.
« Où est SQLite ? »	Dans `db.py` et `data/concerts.db` ; les favoris, préférences, interactions et traces y sont écrits.
« Pourquoi une confirmation ? »	Pour éviter qu'un agent modifie la base sans accord ; la barrière est dans le code Python.
« Les données sont-elles réelles ? »	Celles de Ticketmaster oui ; celles marquées `demo` sont des exemples clairement signalés.
« Pourquoi pas de notifications ? »	Ce n'est pas implémenté ; c'est une perspective, pas une fonctionnalité livrée.
« Que se passe-t-il sans LLM ? »	Le mode règles prend le relais ; les tools et les tests restent utilisables.
« Combien de tests ? »	15 tests automatiques, sans réseau, sur les scénarios, les tools, SQLite et Spotify.
« Pourquoi Ticketmaster ? »	API accessible, liens de billetterie et données françaises ; la couverture reste partielle.
« Pourquoi je ne vois pas tous mes artistes ? »	La recherche large interroge jusqu'à 40 artistes et affiche 30 concerts maximum ; un avertissement signale la limite. Nommer un artiste permet une recherche ciblée.
« Pourquoi SQLite ? »	Simple, local, sans serveur, suffisant pour un projet mono-utilisateur.

Ce qu'il ne faut pas dire

- « L'application envoie des alertes toutes les cinq minutes. » → **Faux**, ce n'est pas implémenté.
- « Bandsintown est intégré. » → **Faux**, c'est une perspective.
- « Les concerts de démonstration sont réels. » → **Faux**, ils sont étiquetés `demo`.
- « Le LLM ne peut jamais se tromper. » → **Trop fort** : les tools et les validations réduisent le risque, ils ne suppriment pas toute erreur.
- Ne pas montrer `.env` ni les clés API pendant la démonstration.

Checklist avant l'oral

- [ ] `run.bat` lancé, page `http://127.0.0.1:5000` ouverte.
- [ ] Base nettoyée (favoris et historique vides).
- [ ] Démo 1 : concerts affichés (réels ou démo signalés).
- [ ] Démo 2 : aucun résultat, aucune invention.
- [ ] Démo 3 : confirmation visible, favori ajouté après Oui.
- [ ] Variante Non : aucun favori ajouté.
- [ ] Fiche d'un favori ouverte et expliquée.
- [ ] Traces visibles.
- [ ] Badge LLM / données / Spotify compris.
- [ ] Tests lancés une fois : `Ran 15 tests ... OK`.
- [ ] Plan B connu si le réseau ou un service tombe.